

Acme Technologies - Engineering Handbook

Version: 2026.1

Document owner: Engineering

Effective date: January 1, 2026

Internal enterprise knowledge base document for LoreForge demonstrations.

Table of Contents

1. Engineering Principles
2. Git Workflow
3. Branching Strategy
4. Pull Requests
5. Code Reviews
6. Testing Expectations
7. CI/CD
8. Production Deployments
9. Logging
10. Monitoring
11. Related Documents

1. Engineering Principles

Engineering at Acme Technologies is organized around reliability, security, maintainability, and explainability. Teams should prefer small changes, typed interfaces, automated verification, and clear operational ownership.

Systems that handle customer data must follow the Acme Technologies Information Security Policy. Systems that expose production health or readiness behavior must follow the Acme Technologies Cloud Operations Runbook.

2. Git Workflow

All production code changes must be tracked in Git. Work should begin from the current main branch unless a release branch is active.

Standard workflow:

1. Create a short-lived branch.
2. Make a focused change.
3. Run local verification.
4. Open a pull request.
5. Address review feedback.
6. Merge only after required checks pass.

Developers must not rewrite shared branch history unless the team has explicitly approved the cleanup. Secrets, credentials, customer documents, and private keys must never be committed.

3. Branching Strategy

Acme Technologies uses trunk-based development with short-lived feature branches.

Branch naming convention:

- 'feature/<short-description>' for product work.
- 'fix/<short-description>' for defects.
- 'ops/<short-description>' for deployment and runtime changes.
- 'docs/<short-description>' for documentation-only changes.

Branches should normally live less than five business days. Long-running work must be split behind safe interfaces or feature flags approved by Engineering.

4. Pull Requests

Pull requests should be small enough for a reviewer to understand in one sitting. Each pull request must include:

- Purpose of the change.
- User or operational impact.
- Verification performed.
- Risk and rollback notes for production-facing changes.
- Screenshots for visible frontend changes.

Pull requests that affect authentication, authorization, encryption, production deployment, logging, or customer data handling require Security review when the change is material.

5. Code Reviews

Reviewers are responsible for correctness, maintainability, security, and operational safety. Reviews should focus on behavior and risk before style.

A reviewer should verify:

- The change is scoped to the stated objective.
- Tests cover important behavior and edge cases.
- Interfaces remain stable unless a breaking change is approved.
- Errors are safe for users and useful for operators.
- Logs do not expose secrets or customer data.
- Ownership and access boundaries are preserved.

Authors should respond to review comments with either a change, a clarifying explanation, or a documented follow-up.

6. Testing Expectations

Every code change must include appropriate verification. The default expectation is automated tests plus local quality checks.

Required checks before merge:

- Unit tests for isolated business logic.
- Integration tests for important request or repository flows.
- Type checking for typed modules.
- Formatting and linting.
- Manual smoke testing when user-visible behavior changes.

Tests should be deterministic. Network calls, paid provider calls, and model downloads must be isolated behind opt-in smoke tests or test doubles.

7. CI/CD

CI must run on pull requests and main branch updates. A pull request cannot merge until required backend and frontend checks pass.

The default pipeline should include:

- Backend test suite.
- Backend formatting check.
- Backend lint check.
- Backend type check.

- Frontend dependency installation from the lock file.
- Frontend tests.
- Frontend linting.
- Frontend production build.

CI should not require live Gemini, OpenRouter, or production database credentials. Live provider smoke tests are separate opt-in checks.

8. Production Deployments

Production deployments require:

- Approved pull request.
- Passing CI.
- No open critical or high severity security review findings.
- Change owner identified.
- Rollback plan documented.
- Maintenance window approval when customer impact is possible.

Database migrations must be reviewed for backward compatibility. Deployment owners must verify '/health' and '/ready' after startup. For infrastructure dependencies, follow the Cloud Operations Runbook.

Emergency production changes require incident commander approval during a SEV-1 or SEV-2 incident and must be followed by a retrospective pull request or postmortem note.

9. Logging

Logs should help operators understand what happened without exposing sensitive data.

Logging requirements:

- Include request IDs when available.
- Use structured fields for stage, status, and duration.
- Avoid passwords, tokens, API keys, customer document contents, prompts, and personal data.
- Avoid high-cardinality labels in metrics and dashboards.
- Log provider failures with safe summaries only.

Security logging requirements are governed by the Information Security Policy.

10. Monitoring

Production services must expose health and readiness signals where applicable.

Applications should provide:

- Liveness endpoint for process health.
- Readiness endpoint for startup completion.
- Request success and failure counters.
- Duration observations for critical workflows.
- Error logs with request IDs and safe failure categories.

The Cloud Operations Runbook defines alert handling, backup expectations, disaster recovery, and maintenance windows.

11. Related Documents

Engineers should use this handbook together with:

- Acme Technologies - Information Security Policy.
- Acme Technologies - Incident Response Playbook.
- Acme Technologies - Cloud Operations Runbook.
- Acme Technologies - Employee Handbook.